

1. SQL INJECTION

Background

After receiving REST API from the console, I run <http://localhost:3003/api/products/:id>, where `:id` is replaced by any given number.

Alternatively: <http://localhost:3003/api/products/1 OR 1=1> to return all results

Note: Injecting `OR 1=1` into the `:id` parameter caused the API to return the full products table. This shows that the backend query is unsanitized and not parametrized.

The information behind the code:

- number of columns: 4
- sequence of columns: id, description, name, price
- data types:
 - id – integer
 - description – text
 - name – text
 - price – integer

Practical significance: from now on all search parameters have to be contained to 4 values within the URL when combining with UNION command

1. Get the REST interface to return a list of all tables and views.

Terminal: `curl "http://localhost:3003/api/products/1%20UNION%20SELECT%20name,NULL,NULL,NULL%20FROM%20sqlite_master%20WHERE%20type%20IN%20('table','view')"`

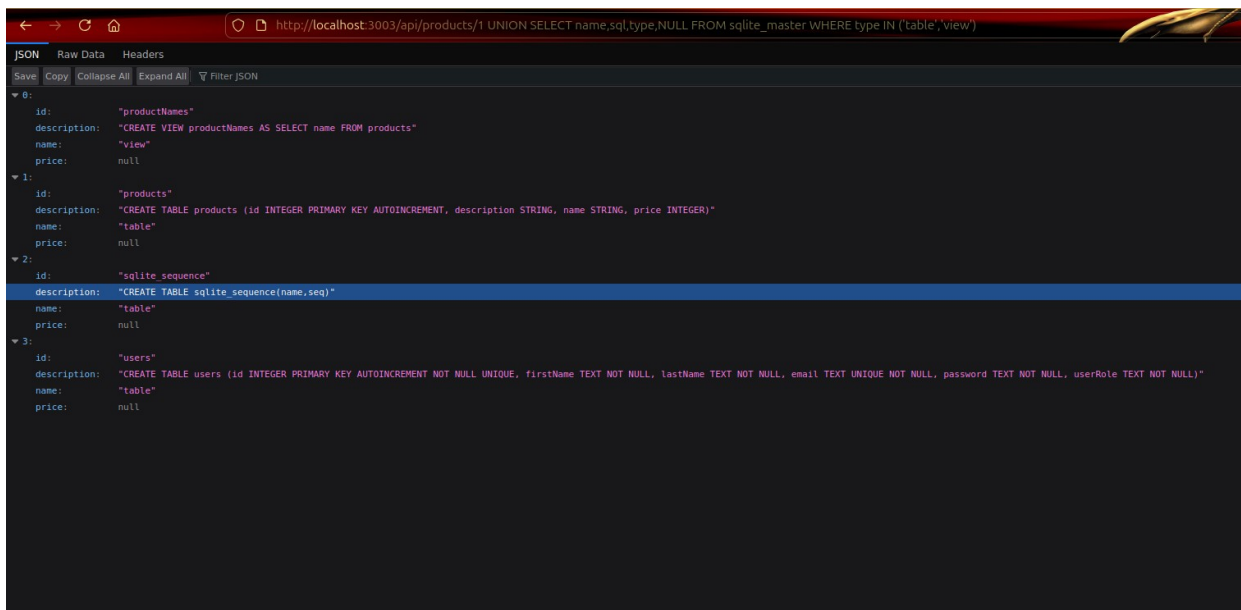
Browser: [http://localhost:3003/api/products/1 UNION SELECT name,sql,type,tbl_name FROM sqlite_master WHERE type IN \('table','view'\)](http://localhost:3003/api/products/1 UNION SELECT name,sql,type,tbl_name FROM sqlite_master WHERE type IN ('table','view'))

*sqlite_master = sqlite_schema

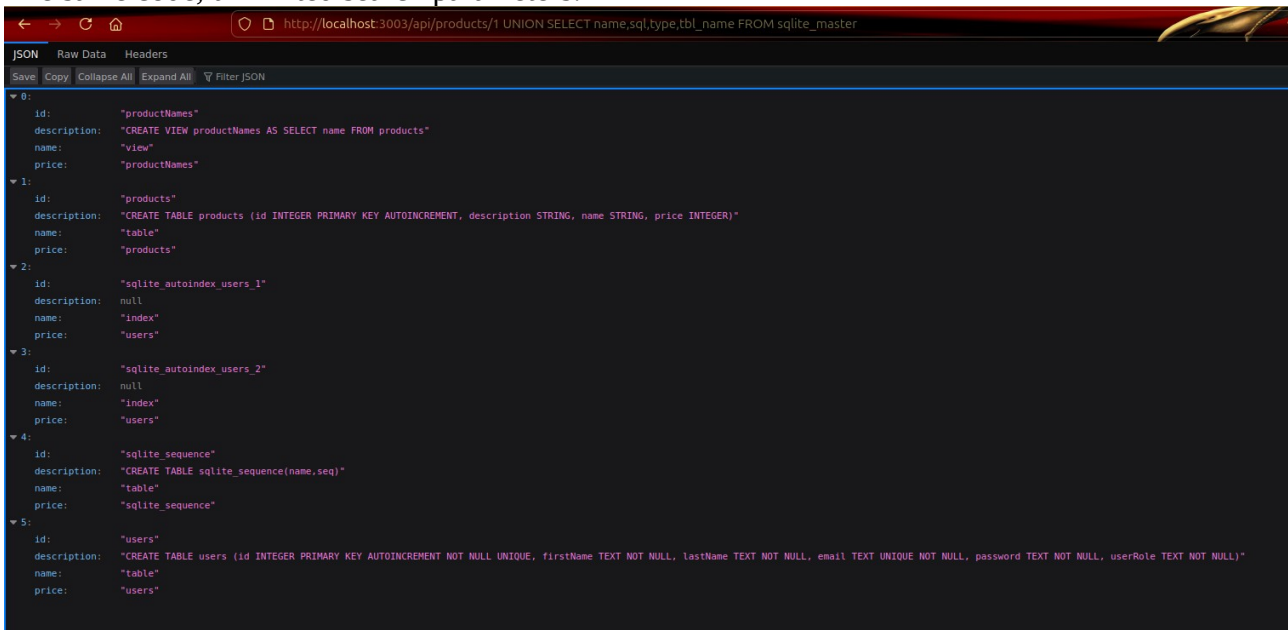
Logic

- the number of columns must be the same as previously discovered number of columns in products table
- the data is extracted via `sqlite_schema` which is where SQLite stores information about itself
- from SQLite's own documentation (<https://sqlite.org/schematab.html>) we know that every SQLite database contains the following metadata: type, name, tbl_name, rootpage, sql
- the approach is based on trial and error and testing what results will return if UNION command forced products API to release the database metadata

Code limited to “table” and “view” parameters:



The same code, unlimited search parameters:



2. Get the REST interface to return a list of all columns and data types in all tables.

Terminal: `curl "http://localhost:3003/api/products/1%20UNION%20SELECT%20name,type,NULL,NULL%20FROM%20pragma_table_info('users')"`

Browser: [http://localhost:3003/api/products/1 UNION SELECT name,type,NULL,NULL FROM pragma_table_info\('users'\)](http://localhost:3003/api/products/1%20UNION%20SELECT%20name,type,NULL,NULL%20FROM%20pragma_table_info('users'))

Use different table names for the pragma value as per results shown in point 1 to see different outcomes for each table

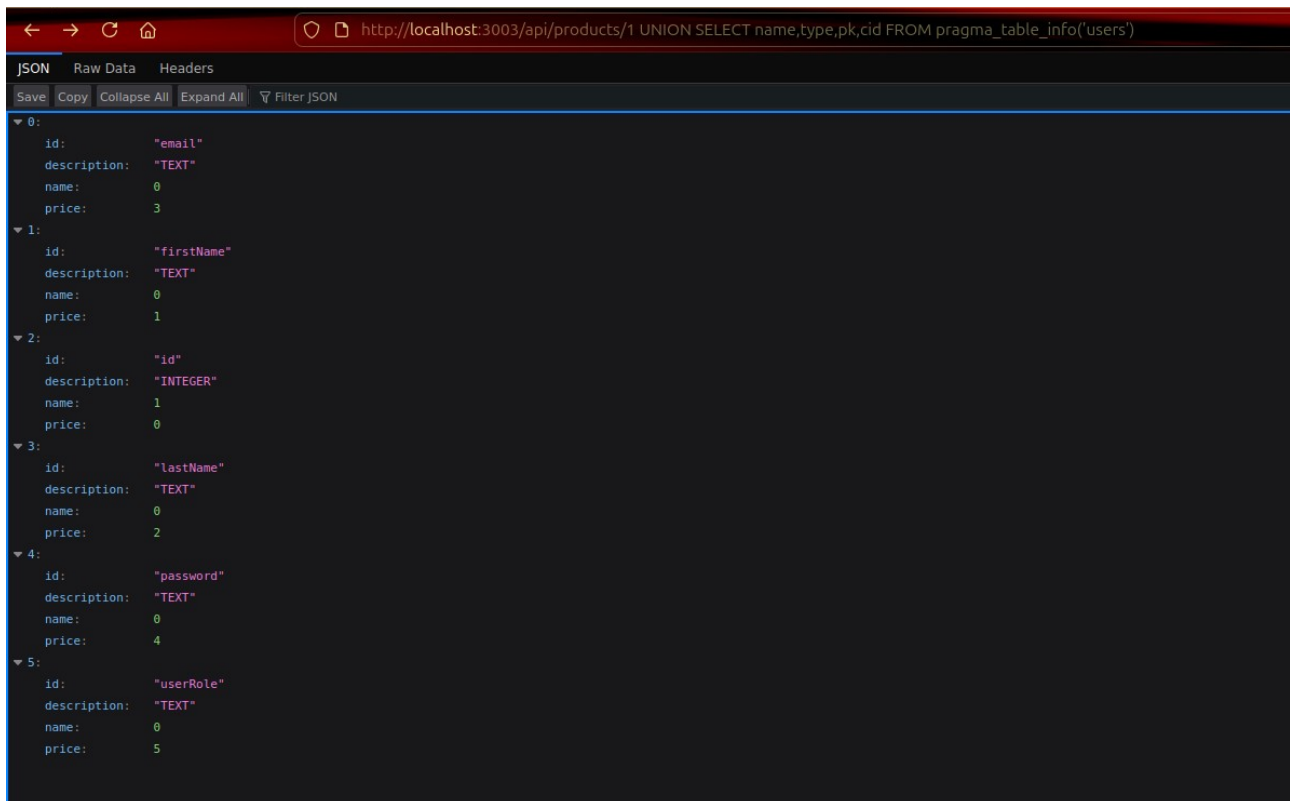
Logic:

→ In SQLi there are two ways to extract information:

1. sqlite_schema → shows built-in tables and metadata of the database
2. PRAGMA table_info(table)- control interface for the database

→ Previous investigation yielded the following tables: “productNames”, “products”, “sqlite_autoindex_users_1”, “sqlite_autoindex_users_2”, “sqlite_sequence”, “users”, so next logical step would be to extract values based on inserted table names (example: “users”)
→ As per SQLite documentation (<https://sqlite.org/pragmas.html>) the values assigned to pragma_table_info are: cid, name, type, notnull, pk, dflt_value.

Further information points: “Users” table shows the following values “email”, “firstName”, “id”, “lastName”, “password”, “userRole”



3. Get the REST interface to return a list of all users in the database.

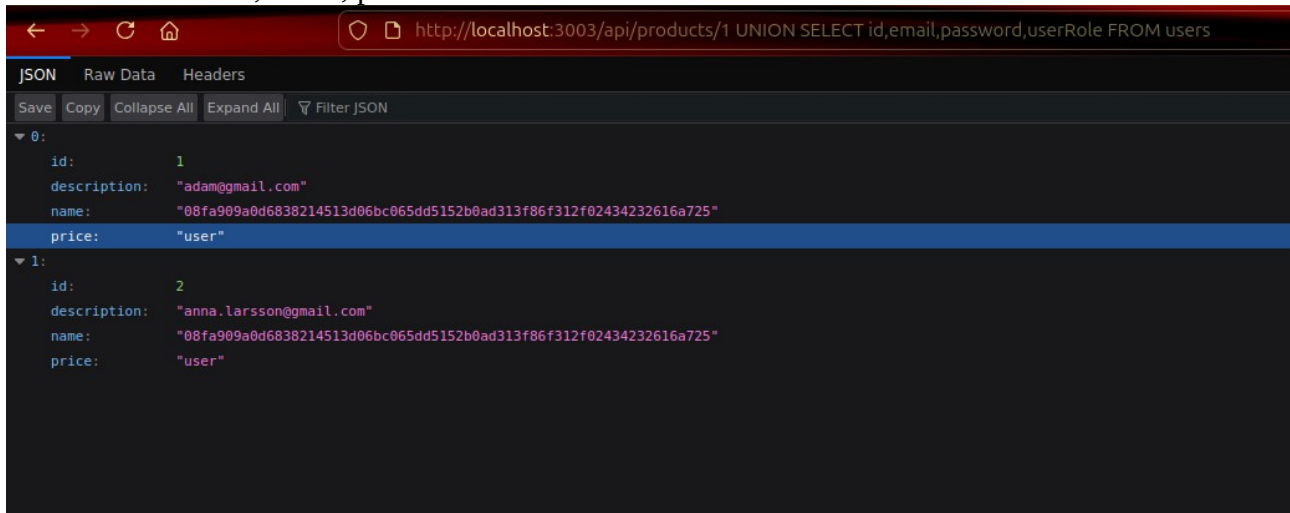
Terminal: `curl "http://localhost:3003/api/products/1%20UNION%20SELECT%20id,email,password, userRole%20FROM%20users"`

Browser: <http://localhost:3003/api/products/1 UNION SELECT id,email,password,userRole FROM users>

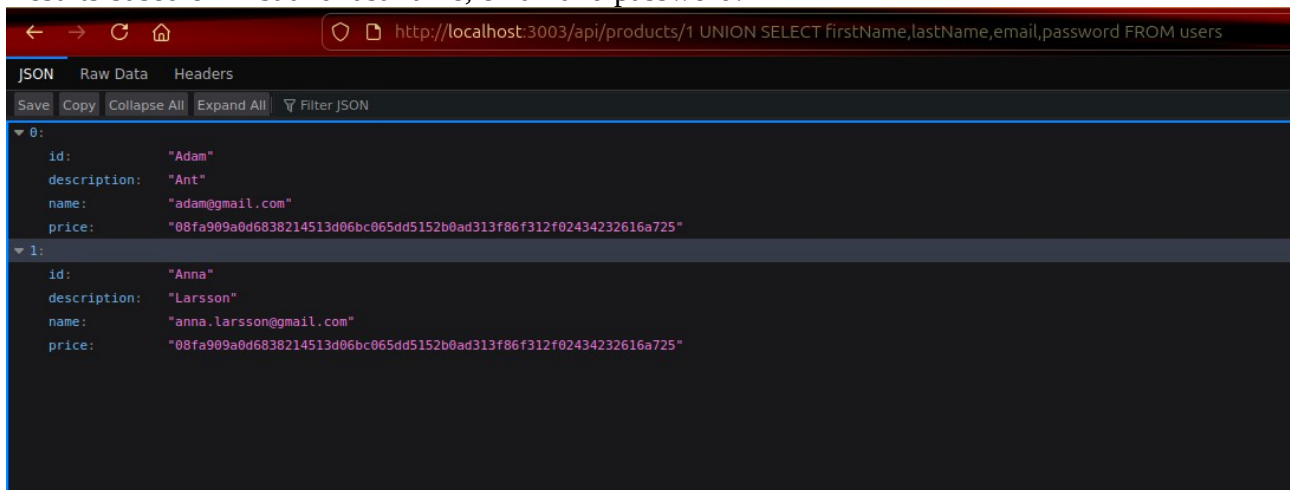
Logic:

→ previous search has shown 6 results pertaining to “users” table, by inserting 4 of them we can find out specific data about existing users

Results based on id, email, password and user role:



Results based on first and last name, email and password:



There are currently 2 users listed on the website

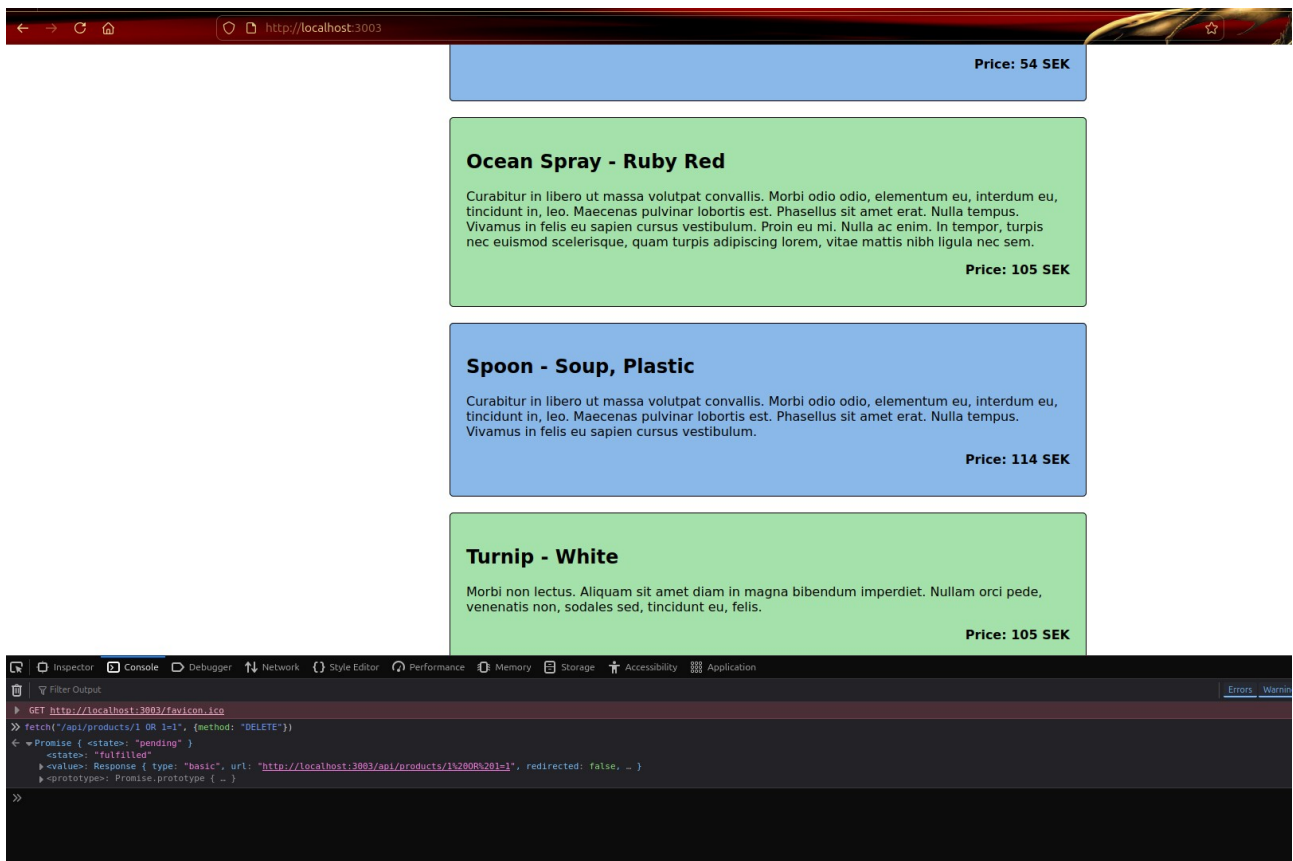
4. Get the REST interface to erase all products with a single query.

Browser address bar can only send GET requests, not DELETE

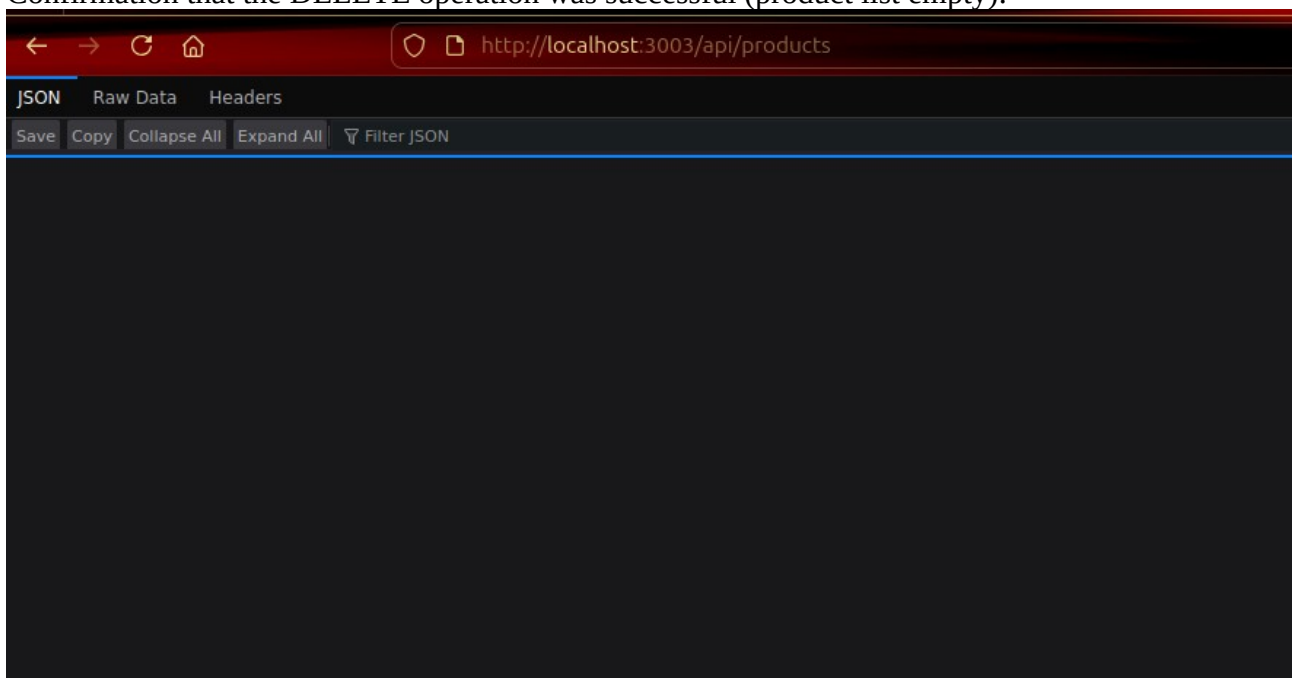
Terminal: `curl -X DELETE "http://localhost:3003/api/products/1%20OR%201=1"`

VS Code → Thunder Client: set method DELETE → execute: `http://localhost:3003/api/products/1 OR 1=1`

Browser → Dev Tools → Console → `fetch("/api/products/1 OR 1=1", { method: "DELETE" })`



Confirmation that the DELETE operation was successful (product list empty):



Logic:

- The code runs the equivalent of the command: DELETE products WHERE id = <id input> or true/everything”
- All items in products table are being removed
- alternatively can’t DROP TABLE on REST API or without causing a syntax error.

Which category in the OWASP Top 10 belongs to the vulnerability?

OWASP A05:2025 – Injection. The name is literal: SQL code injection.

How did you secure the code? Show what you changed and explain why it works.

The following changes were made into the code

1. GET /api/products/:id

ORIGINAL CODE:

```
app.get('/api/products/:id', (req, res) => {  
  req.body = { id: req.params.id };  
  run(res, `  
    SELECT * FROM products  
    WHERE id = ${req.params.id}  
  `, 'all');});
```

CHANGED CODE:

```
app.get('/api/products/:id', (req, res) => {  
  const id = req.params.id;  
  run(res, `  
    SELECT * FROM products  
    WHERE id = ?  
  `, 'all', [id]);});
```

`${req.params.id}` is directly concatenated into SQL query string, when database parses the query it becomes a command, not data so hackers can modify the command logic. Using `?` As a placeholder tells SQLi to treat the input as a text value, not code. `[id]` binds it.

1. DELETE /api/products/:id

ORIGINAL CODE

```
app.delete('/api/products/:id', (req, res) => {  
  run(res, `  
    DELETE FROM products  
    WHERE id = ${req.params.id}`);});
```

CHANGED CODE

```
app.delete('/api/products/:id', (req, res) => {  
  const id = req.params.id;  
  run(res, `  
    DELETE FROM products  
    WHERE id = ?  
  `, 'run', [id]); });
```

It prevents deleting the entire value by injecting `1 OR 1=1`. Also a `?` placeholder is added as in the example above.

3. RUN helper function

ORIGINAL CODE

```
function run(res, query, type = 'run') {  
  res.json(db.prepare(query)[type]());}
```

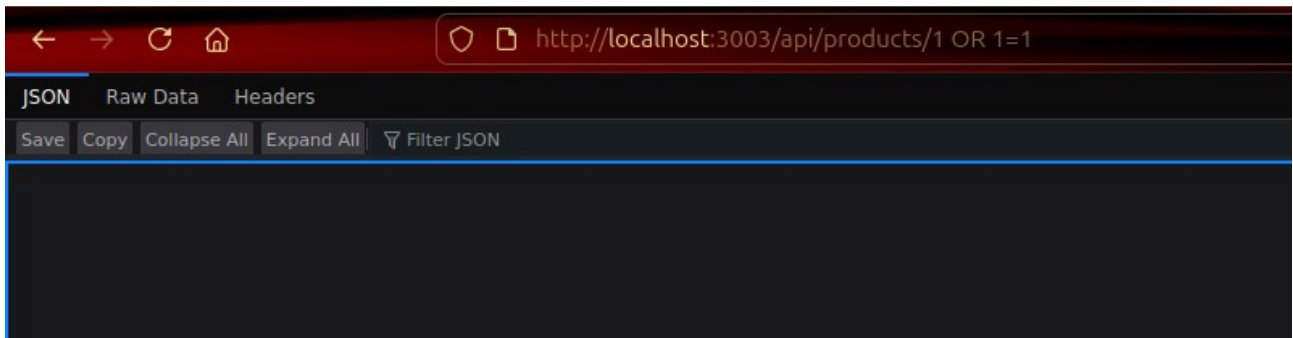
CHANGED CODE

```
function run(res, query, type = 'run', params = []) {  
  res.json(db.prepare(query)[type](...params)); }
```

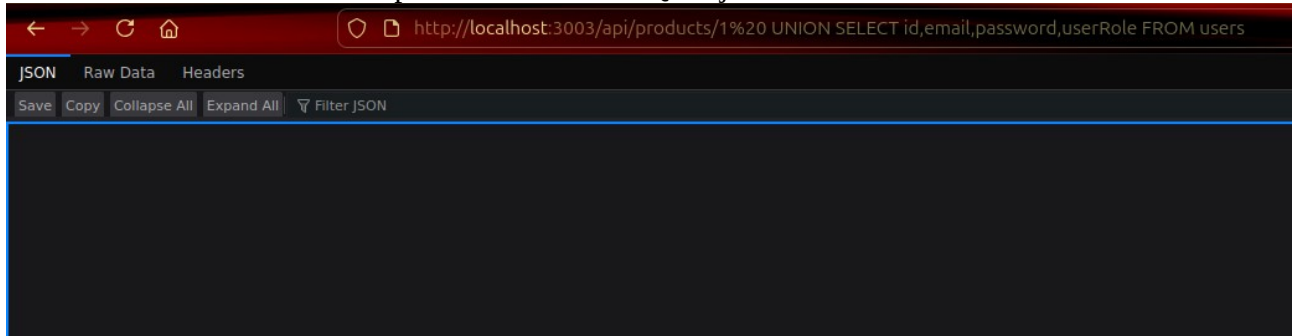
Allows passing parameters to placeholders (?) and allows the run() function to safely pass user-supplied values to both the GET and DELETE routes using prepared statements.”

Changes after new code has been deployed

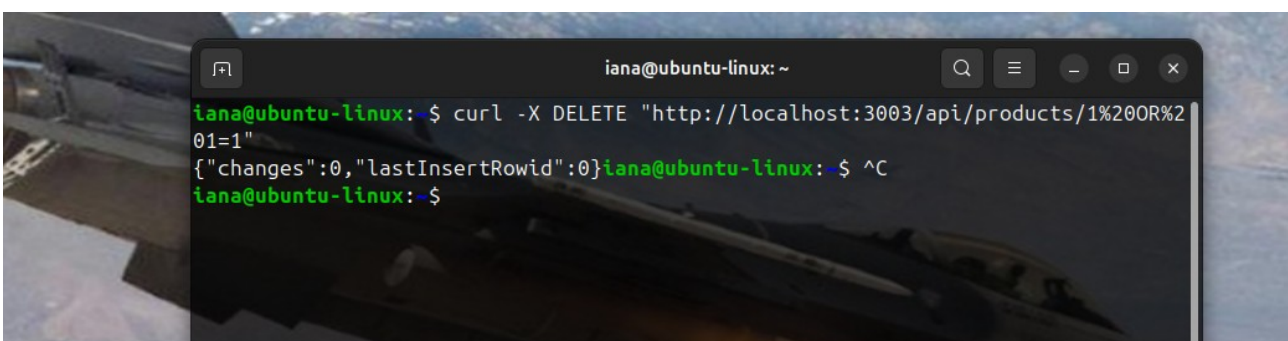
Screenshot shows empty products list upon entering <http://localhost:3003/api/products/1 OR 1=1> (a new copy of the file was loaded on Visual Code). After implementing prepared statements, attempts to inject SQL via the link above are treated as literal input rather than executable SQL and the database returns either a single product or no results instead of executing the injected query.



Screenshot shows failed attempt to list users via SQL injection



Another screenshot shows that implementing prepared statements in the code blocked the attempt to delete the database



Code after changes

```
const db = betterSqlite3('products.db');
const app = express();
const port = 3003;

app.use(express.static('www'));

app.use(express.json());

app.get('/api/products', (req, res) => {
  run(res, `
    SELECT * FROM products
  `, 'all');
});

app.get('/api/products/:id', (req, res) => {
  const id = req.params.id; //get user supplied id
  req.body = { id: req.params.id };
  run(res, `
    SELECT * FROM products
    WHERE id = ?
  `, 'all', [id]);
}); // add placeholders for prepared statements for all user input

app.delete('/api/products/:id', (req, res) => {
  const id = req.params.id; //get the user supplied id
  run(res, `
    DELETE FROM products
    WHERE id = ?
  `, 'run', [id]); //add placeholders for prepared statements for all user input
});

function run(res, query, type = 'run', params = []) {
  res.json(db.prepare(query)[type](...params)); // added params array to pass user inputs to prepared statements
}

app.listen(port,
  () => console.log('Listening on http://localhost:' + port));
```

Risk assessment

SQL injection as presented in this task should be considered high to critical risk.

The main reason why it's dangerous it's because the attack is easy to discover and exploit- the REST API exposes user input directly in the URL (`/api/products/:id`) and the value is inserted into the SQL query without validation or parameterization. A hacker only needs basic SQL knowledge to try simple commands like `OR 1=1` or `UNION SELECT`.

The technical impact is serious. During testing it was possible to:

- read sensitive data from other tables (i.e. emails and passwords),
- find out database's structure (tables and columns),
- delete all products from the database with a single request.

If an attack can expose private data, change or damage it or make the infrastructure unusable by deleting or corrupting data then it affects confidentiality and integrity of the system.

To fix it, the code was changed to use prepared statements with placeholders (?). This prevents user input from being interpreted as part of the SQL logic because the database treats it as data only.

However, this fix alone should not be the only defense mechanism.

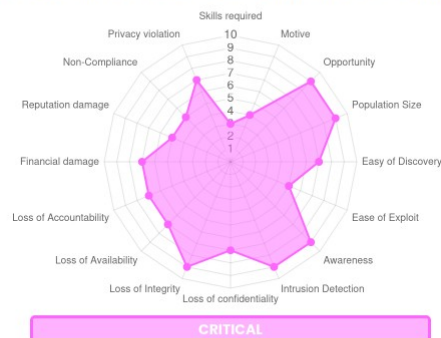
A better approach is defense in depth, meaning multiple security layers. For example:

1. input validation only allowing numeric IDs

2. proper error handling so database errors are not revealed to users
3. authentication/authorization for sensitive AP
4. adding logging and monitoring features to detect suspicious requests.

Another thing is that even with prepared statements, other parts of the application could still be vulnerable if developers later introduce other features and pieces of code. Long term it means that security depends not only on single fix but also on secure coding practices across the entire design. Overall, because the vulnerability is easy to exploit and can expose or destroy data, the OWASP risk rating should be considered high to critical

OWASP Risk Assessment Calculator



Threat Agent Factors		Technical Impact Factors	
Skill level	Some technical skills (3)	Loss of confidentiality	extensive critical data disclosed (7)
Motive	Possible reward (4)	Loss of integrity	All data totally corrupt (9)
Opportunity	No access or resources required (9)	Loss of availability	Extensive primary services interrupted (7)
Size	Anonymous Internet users (9)	Loss of accountability	Possibly traceable (7)

Vulnerability Factors		Business Impact Factors	
Ease of discovery	Easy (7)	Financial damage	Significant effect on annual profit (7)
Ease of exploit	Easy (5)	Reputation damage	Loss of goodwill (5)
Awareness	Public knowledge (9)	Non-compliance	Clear violation (5)
Intrusion detection	Not logged (9)	Privacy violation	Thousands of people (7)

Likelihood score	Impact score
6.875 HIGH	6.750 HIGH